# RACIOCÍNIO AUTOMÁTICO

Luís Morgado

ISEL-DEI

# RESOLUÇÃO AUTOMÁTICA DE PROBLEMAS

A resolução automática de problemas é uma área da inteligência artificial orientada para a resolução de problemas por meios algorítmicos (máquinas computacionais), através de **raciocínio automático**

## Exemplos:



Planeamento robótico



Planeamento de sistemas logísticos



Controlo de personagens multimédia




Navegação autónoma

$$y = c_0 + c_2x^2 + c_4x^4 + c_6x^6 + \cdots$$

$$= c_0 + \frac{c_0}{2}x^2 + \frac{c_0}{2\cdot 4}x^4 + \frac{c_0}{2\cdot 4\cdot 6}x^6 + \cdots$$

$$= c_0\left(1 + \frac{1}{2}x^2 + \frac{1}{2\cdot 4}x^4 + \frac{1}{2\cdot 4\cdot 6}x^6 + \cdots\right)$$

Resolução de problemas matemáticos



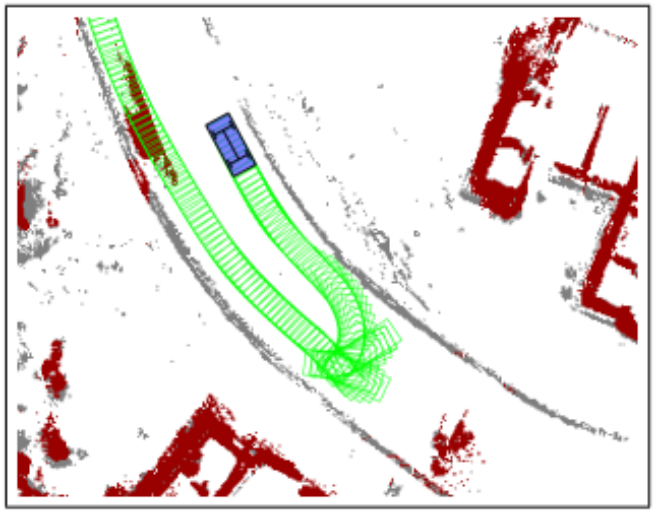
Jogos diversos

# RESOLUÇÃO AUTOMÁTICA DE PROBLEMAS

## Tipos de problemas

- **Problemas de planeamento**
  - Problemas cuja solução consiste numa sequência de acções a realizar e de situações a percorrer para, partindo de uma situação inicial, atingir uma situação final (objectivo)
  - **Resultado**:
    - Sequência de acções e situações (*plano*)

**Exemplo**:
Planeamento da trajectória de um veículo



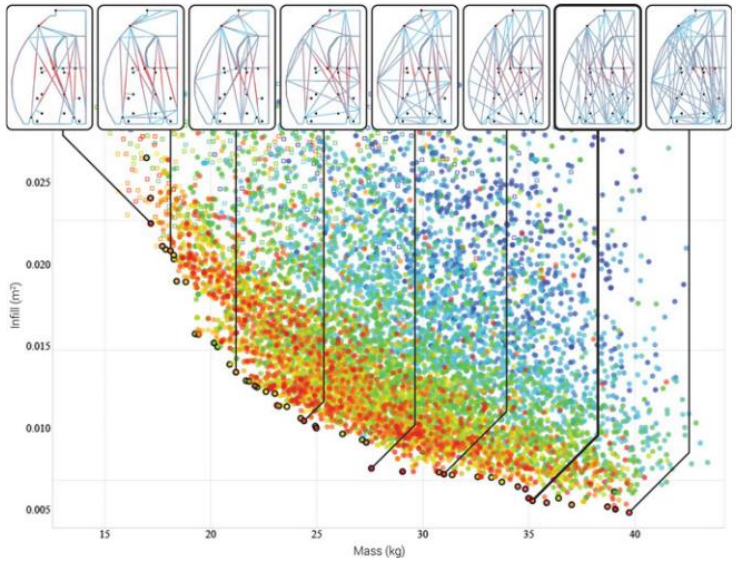
# RESOLUÇÃO AUTOMÁTICA DE PROBLEMAS

## Tipos de problemas (cont.)

- **Problemas de optimização**
  - Problemas em que se pretende encontrar a melhor configuração de parâmetros de modo a maximizar uma função de valor ou de adequação
  - **Resultado**:
    - Configuração de parâmetros (*estado*)

[Nagy *et al.*, 2018]

**Exemplo:**
Optimização da configuração de um painel composto por vários segmentos de modo a maximizar a resistência e a minimizar o peso



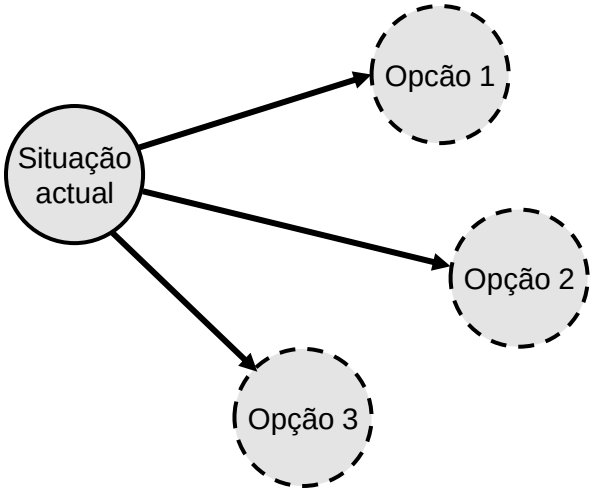
# RACIOCÍNIO AUTOMÁTICO

- Capacidade de um sistema computacional resolver de forma automática um problema com base numa representação de conhecimento do respectivo domínio, produzindo uma solução a partir de diversas alternativas possíveis
- Processo computacional que tendo como entrada uma **representação de conhecimento** de um determinado domínio, **produz como resultado conclusões baseadas nesse conhecimento**
- O processo de manipulação da representação de forma a obter conclusões é normalmente designado ***inferência***

# RACIOCÍNIO AUTOMÁTICO

No caso geral, o processo de raciocínio automático envolve dois tipos de actividades principais, a *exploração de opções* possíveis e a *avaliação de opções* para decisão acerca das melhores opções

- **Exploração de opções**
  - **Raciocínio prospectivo**
    - Antecipação do que pode acontecer
  - **Simulação** interna do domínio do problema
    - Requer formas de **representação interna** do domínio do problema
- **Avaliação de opções**
  - **Custo**
    - Recursos necessários
  - **Valor**
    - Ganho ou perda, medido por exemplo em termos de *utilidade*



# RACIOCÍNIO AUTOMÁTICO

- **Aspectos principais**
  - **Representação do problema**
    - Determinante para o processo de raciocínio
      - **Eficiência**
      - **Eficácia**
  - **Método de raciocínio**
    - Define método de exploração de alternativas (opções)
    - Define estratégia de controlo do raciocínio
      - Critérios de prioridade de exploração de alternativas

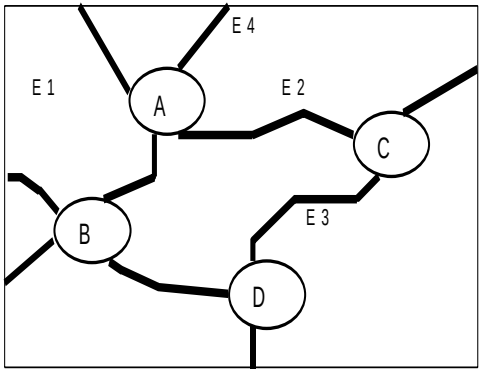
# REPRESENTAÇÃO DO PROBLEMA

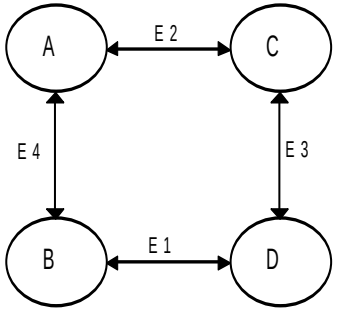
## A forma de representação deve ser adequada

- Ao domínio do problema
- Ao método de raciocínio

**Exemplo**: representação de uma rede de estradas

Mapa




Grafo

Tabela

| | A | B | C | D |
|---|---|---|---|---|
| A | | E 4 | E 2 | - |
| B | E 4 | | - | E 1 |
| C | E 2 | - | | E 3 |
| D | - | E 1 | E 3 | |

# MÉTODOS DE RACIOCÍNIO AUTOMÁTICO

## Exemplos:

- **Inferência lógica**
  - Processo de derivar conclusões a partir de premissas conhecidas, utilizando regras lógicas para chegar a novas informações ou decisões
- **Inferência estatística**
  - Processo de derivar conclusões a partir de amostras de dados disponíveis com base em modelos estatísticos
- **Planeamento simbólico em espaços de planos**
  - Processo de derivar planos de acção com base na exploração de um espaço de planos com base em representações simbólicas de situações e acções
- **Procura em espaços de estados**
  - Processo de derivar soluções através de procura num espaço de estados

# PROCURA EM ESPAÇOS DE ESTADOS

## Exemplo: Problema do puzzle de peças deslizantes

Organizar as peças de um puzzle movendo uma peça de cada vez para a posição vazia




| | | |
|---|---|---|
| 2 | 3 | 7 |
| 5 | 8 | 1 |
| 4 | 6 | |

?

| | | |
|---|---|---|
| 1 | 2 | 3 |
| 4 | 5 | 6 |
| 7 | 8 | |

Como resolver este problema?

Em que consiste uma solução?

# PROCURA EM ESPAÇOS DE ESTADOS

- O conceito de ***estado***
  - Representa uma situação na resolução de um problema ou evolução de um sistema
    - Inclui toda a informação relevante para descrever univocamente qualquer situação na resolução de um problema ou evolução de um sistema

**Exemplo**: Puzzle de peças deslizantes

- Informação de estado: configuração do puzzle




| | | |
|---|---|---|
| 2 | 3 | 7 |
| 5 | 8 | 1 |
| 4 | 6 | |

# PROCURA EM ESPAÇOS DE ESTADOS

- O conceito de ***transição*** de estado
  - Representa uma mudança de estado
    - Representa a realização de uma transformação de estado, por efeito de uma acção, produzindo um novo estado a partir do estado actual

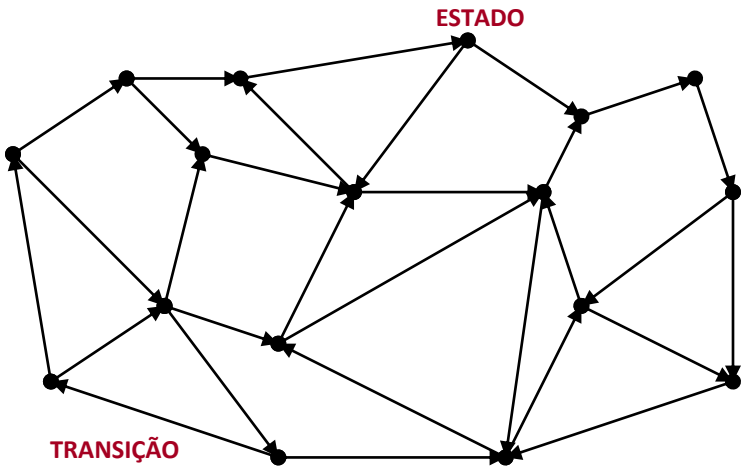
**Exemplo**: Puzzle de peças deslizantes

- Transições de estado: transformações possíveis por efeito de movimentação de peças para a posição vazia



# PROCURA EM ESPAÇOS DE ESTADOS

- O conceito de ***espaço de estados***
  - Representa o conjunto de todos os estados possíveis e das respectivas transições, para um determinado problema ou sistema
  - Pode ser representado como um grafo
    - Vértices representam estados
    - Arcos representam transições



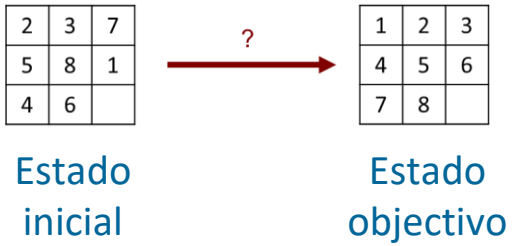
# PROCURA EM ESPAÇOS DE ESTADOS

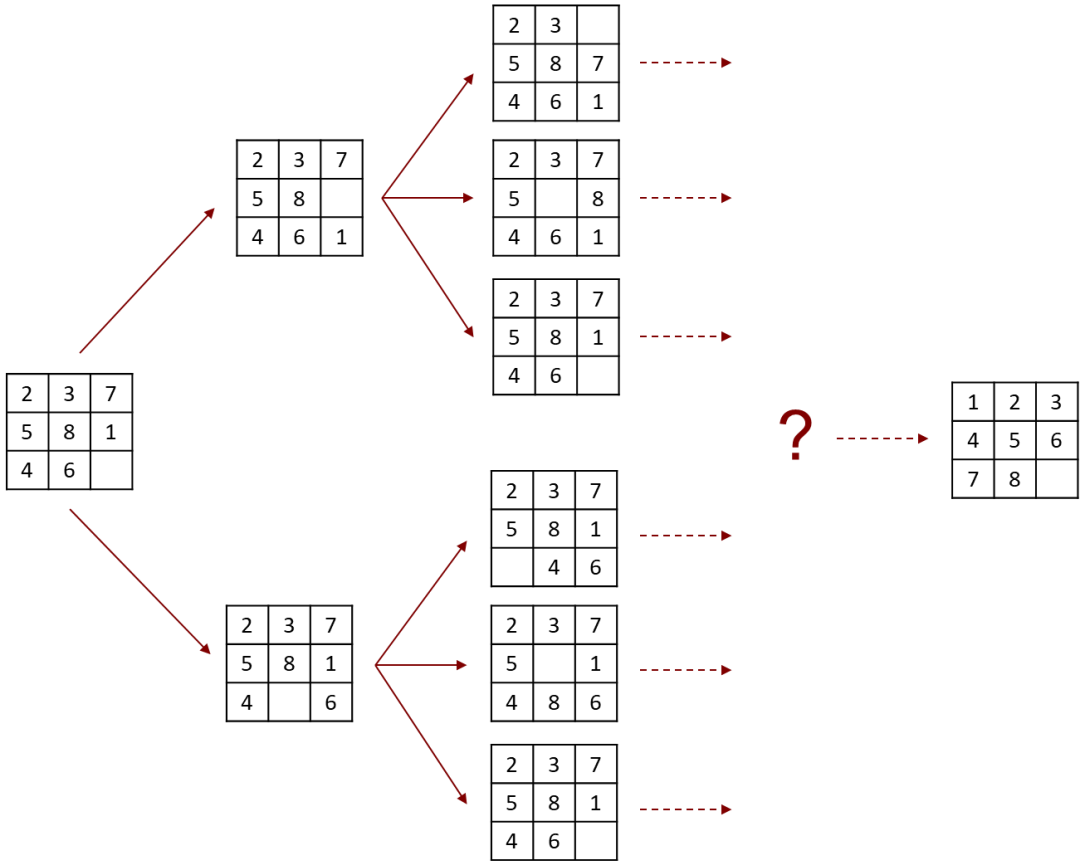
**Resolução de um problema**:

Procura de uma solução por exploração de transições de estado possíveis no espaço de estados

**Exemplo:**

Problema do puzzle de peças deslizantes

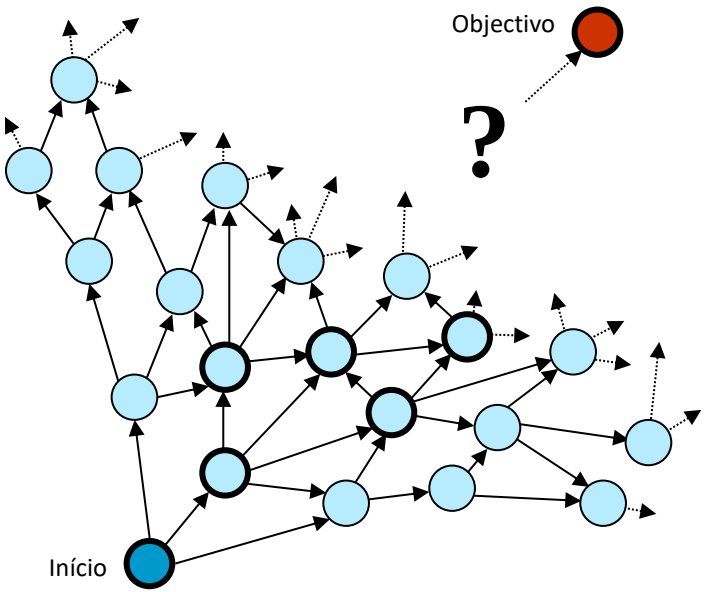


# MÉTODO DE RACIOCÍNIO

## Raciocínio em espaços de estados

- **Raciocínio no sentido directo (*forward*)**
  - O processo de inferência tem início numa situação inicial e opera no sentido de atingir um determinado objectivo
  - Caso geral da procura em espaços de estados

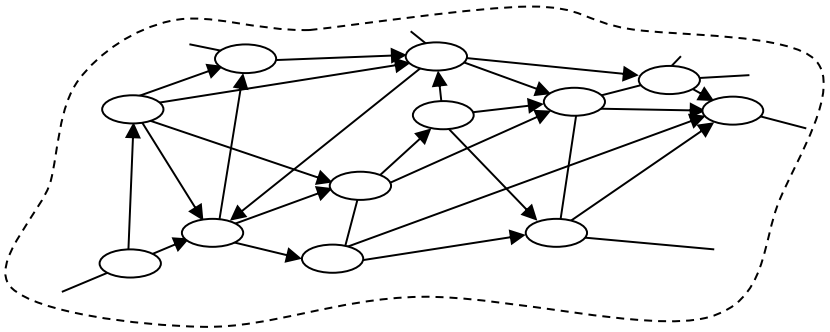



A forma de atingir os objectivos pode envolver um grande número de opções possíveis, cujo número cresce exponencialmente com o número de passos da solução

# RACIOCÍNIO ATRAVÉS DE PROCURA

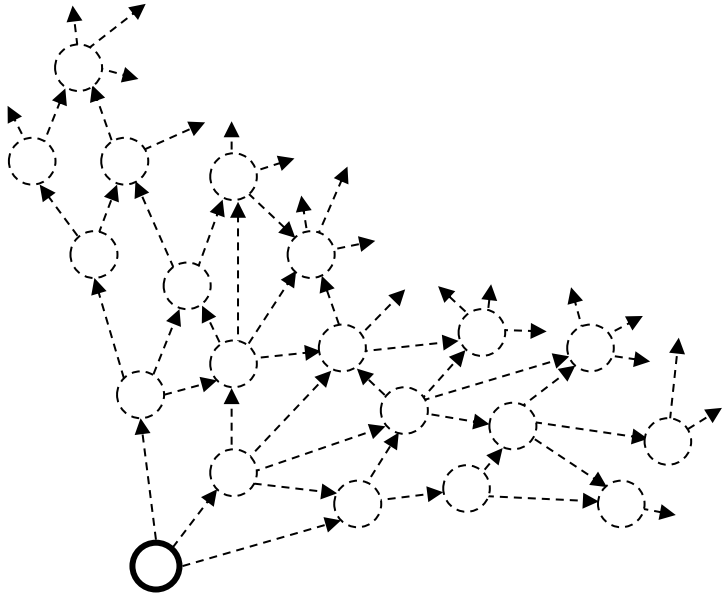
## Procura em espaços de estados

Processo de exploração das configurações (estados) possíveis de um problema, a fim de encontrar uma solução (um percurso no espaço de estados) que concretize um determinado objectivo



**Estado**

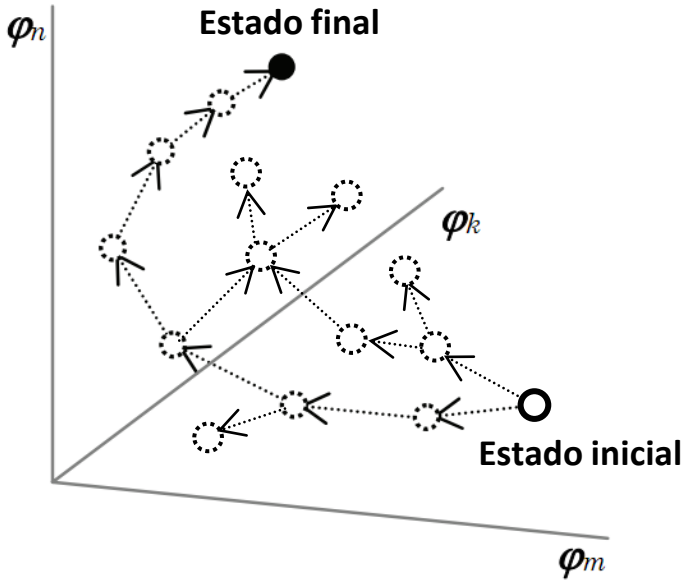
**Transição de estado**



**Simulação de hipóteses de evolução de estado**

# RACIOCÍNIO ATRAVÉS DE PROCURA

- **Estado**
  - Define situação
- **Transição**
  - Define transformação de estado
  - **Operador**
    - Representação de **acção**
- **Problema**
  - Estado inicial
  - Estado final ou função objectivo
  - Operadores
  - Função de avaliação (custo)
- **Solução**
  - Percurso no espaço de estados
    - Para problemas de planeamento
  - Estado final
    - Para problemas de optimização



# RACIOCÍNO ATRAVÉS DE PROCURA

## Método *Gerar-e-Testar*

```
1. Definir objectivo
2. Definir estado inicial
3. Repetir
4.    Comparar estado com o objectivo
5.    Se o objectivo tiver sido atingido então
      terminar e retornar a solução
6.    Aplicar um operador possível ao estado
      actual gerando um novo estado (transição
      de estado)
```

- Não viável no caso geral
  - Procura não sistemática
    - Ineficiente
  - **Problema da explosão combinatória**

# MÉTODOS DE PROCURA EXAUSTIVOS
## (NÃO INFORMADOS)

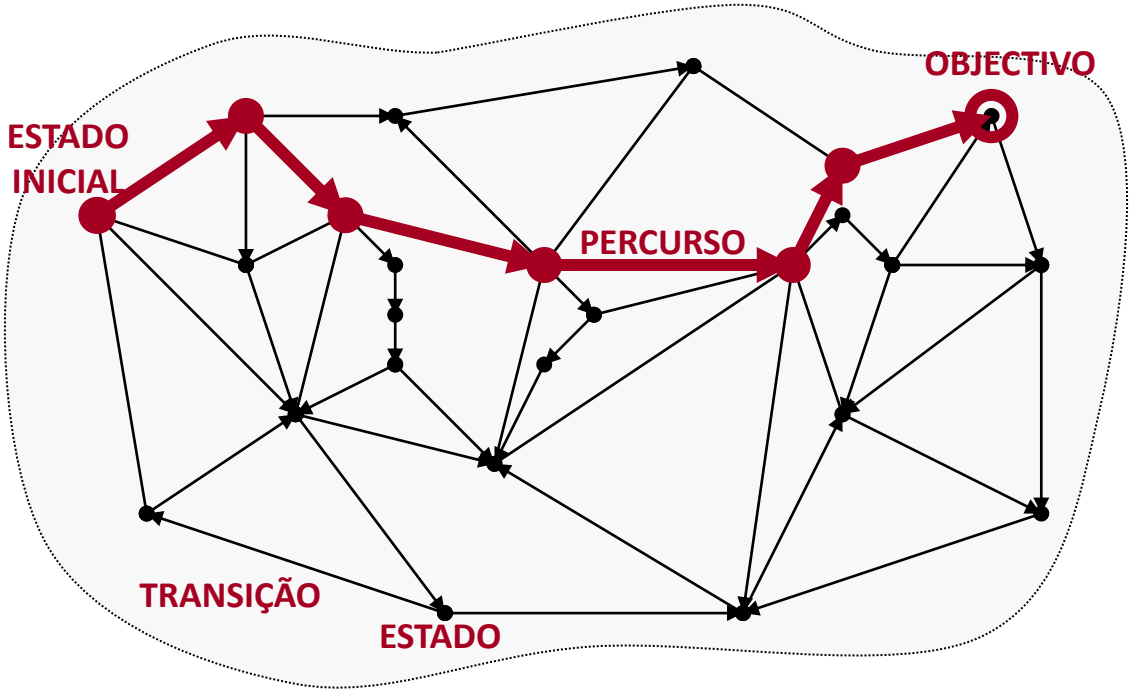
Estratégias de exploração do espaço de estados (*controlo da procura*) que realizam uma procura exaustiva do espaço de estados

- Não tiram partido de *conhecimento do domínio do problema* para guiar a procura
- **Procura não guiada** (**exaustiva**)

# PROCESSO DE PROCURA

O processo de procura ocorre por **exploração do espaço de estados** a partir do estado inicial

O espaço de estados é representado como um grafo, onde cada vértice corresponde a um estado e cada arco corresponde a uma transição entre estados



# PROCESSO DE PROCURA

## Em cada passo de procura:

- É verificado se o **estado actual corresponde ao *objectivo***
  - Se corresponder ao objectivo, o processamento termina e é retornado o percurso (caminho com a sequência de estados e operadores) do estado inicial ao estado objectivo
- Não sendo o estado actual um objectivo, esse estado é ***expandido***, sendo gerados todos os ***estados sucessores*** por aplicação dos vários ***operadores*** possíveis
- Para cada estado sucessor é repetido o processo
- **Se não existirem estados sucessores, o processo de procura termina** com a indicação de que não existe solução
  - A não existência de estados sucessores resulta, por exemplo, da exploração de todo o espaço de estados sem ter sido encontrada solução

# ÁRVORE DE PROCURA

Para **manter a informação gerada em cada passo de procura** é mantida uma estrutura de informação, designada ***árvore de procura***

Consiste numa **estrutura em árvore, organizada em *nós***, que mantém informação relativa a cada transição de estado explorada

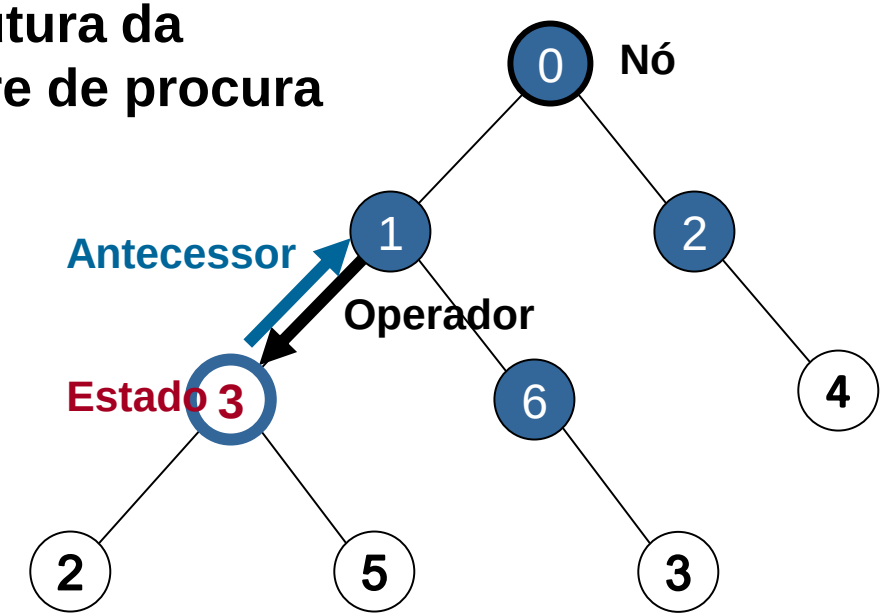
**Relaciona cada nó com o seu antecessor** e mantendo informação do estado correspondente ao nó e do operador que originou a transição de estado respectiva gerando esse novo estado

**Nó:**

- Estado
- Operador (que gerou o estado)
- Antecessor
- Profundidade (do nó)
- Custo (da raiz até ao nó)

**Profundidade** e **custo**:
Informação complementar para controlo do processo de procura

**Estrutura da árvore de procura**

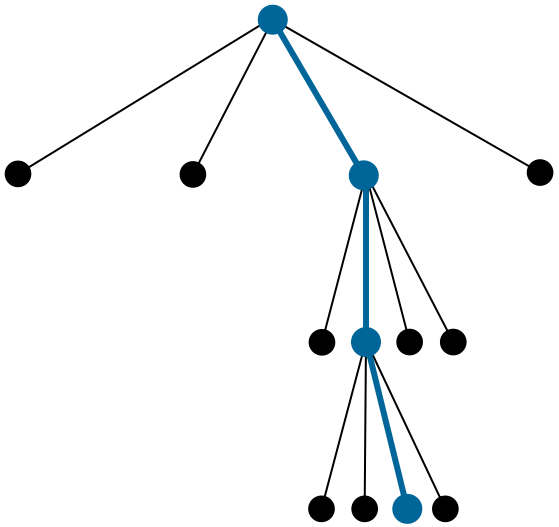


# MÉTODOS DE PROCURA EXAUSTIVOS
## (NÃO INFORMADOS)

- **Procura em profundidade**
  - Critério de exploração:
    - **Maior profundidade**
  - Variantes
    - Procura em profundidade limitada
    - Procura em profundidade iterativa
- **Procura em largura**
  - Critério de exploração:
    - **Menor profundidade**
- **Procura de custo uniforme**
  - Critério de exploração:
    - **Custo da solução**

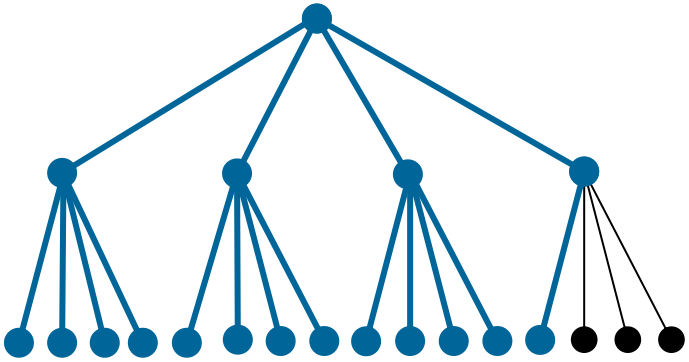
# MÉTODOS DE PROCURA EXAUSTIVOS

## Procura em profundidade



A procura avança através de um caminho único, **explorando primeiro nós de maior profundidade** (os nós mais recentes)

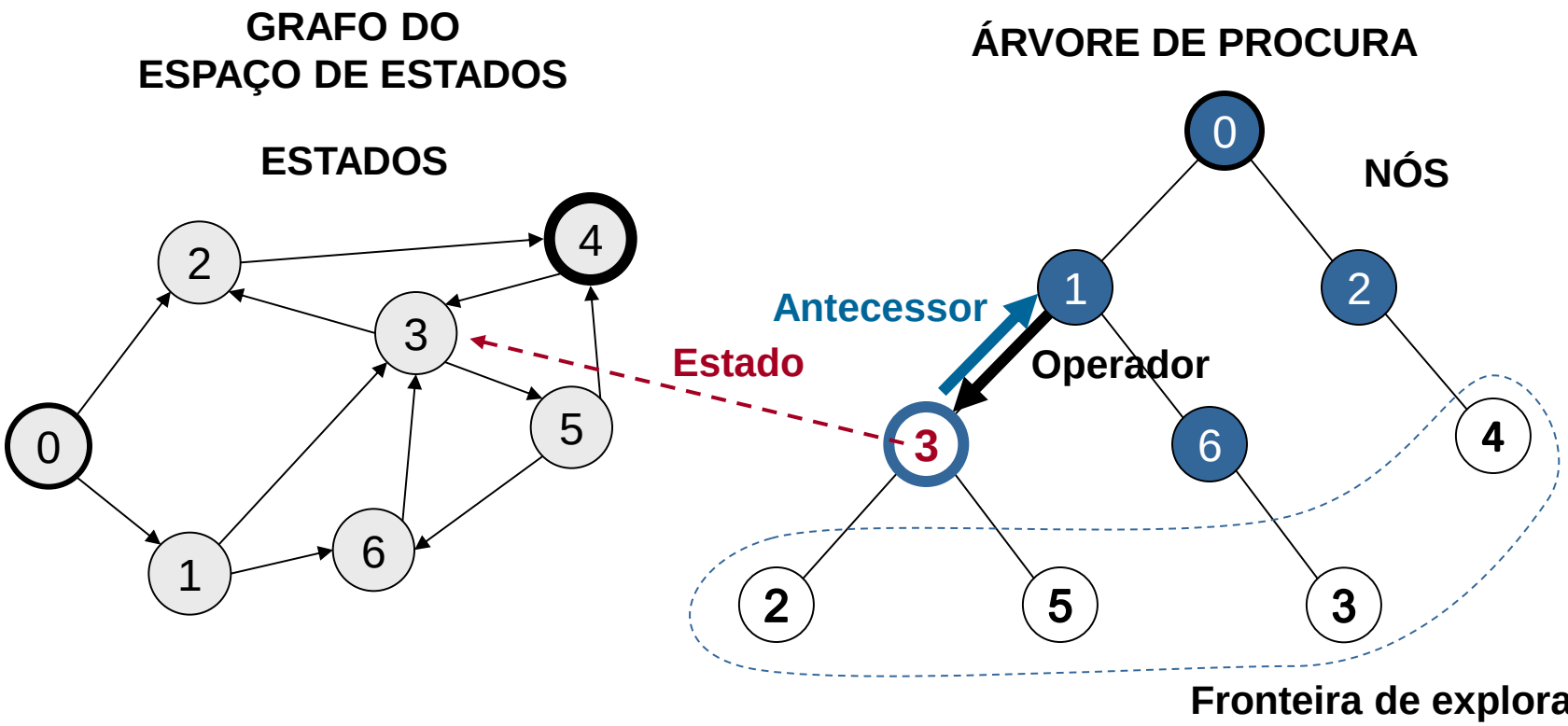
## Procura em largura



A procura avança mantendo múltiplos caminhos alternativos, **explorando primeiro nós de menor profundidade** (os nós menos recentes)

# PROCESSO DE PROCURA GERAL

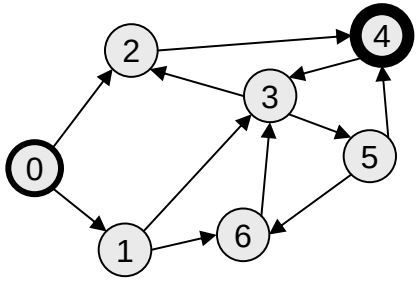
- Exploração sucessiva do espaço de estados
- Etapa de procura: **Nó**
- **Árvore de procura** (possibilita eliminação de redundância entre caminhos)
  - Raiz: Nó correspondente ao e*stado inicial*
- **Fronteira de exploração** (estrutura de dados com relação de ordem)
  - Critério de ordenação determina estratégia de controlo da procura



# MÉTODOS DE PROCURA

## Procura em profundidade (DFS - *Depth-First Search*)

- Estratégia de controlo
  - Explorar primeiro os nós mais **recentes**




Grafo do
Espaço de Estados

Fronteira de exploração [ ]

[0]

0 [ ]

[**1, 2**]

1 [2]

[**3, 6**, 2]

3 [6,2]

[**2, 5**, 6, 2]

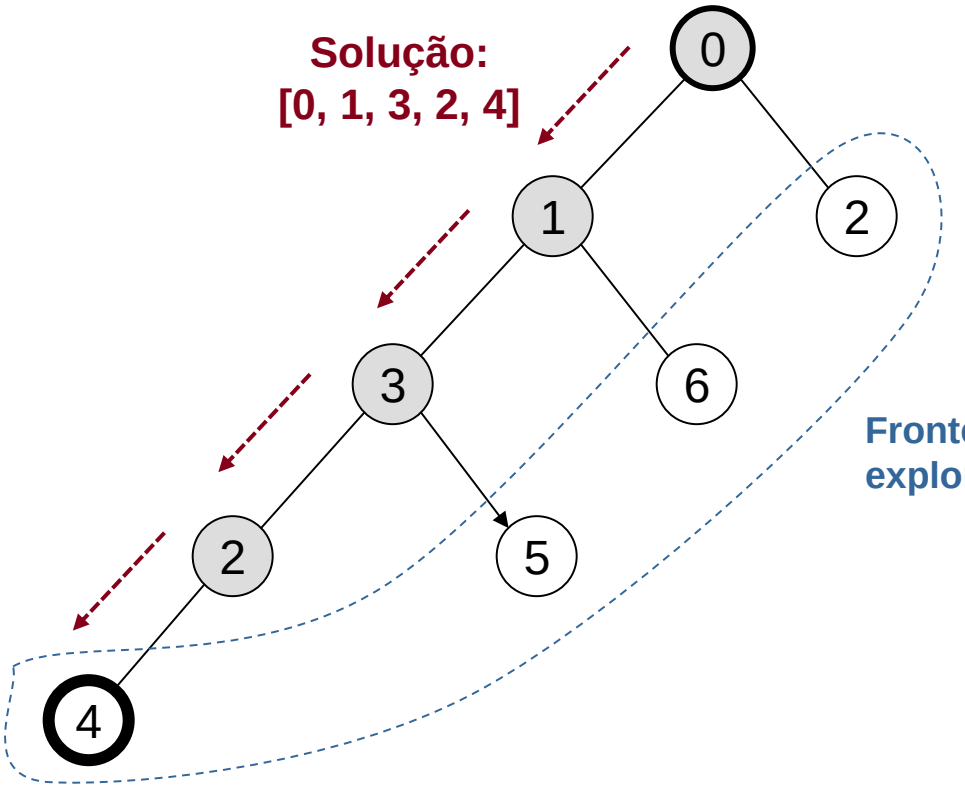
2 [5,6,2]

[**4**, 5, 6, 2]

**Estado objectivo** → **4** [5,6,2]

Árvore de Procura

**Solução:**
**[0, 1, 3, 2, 4]**

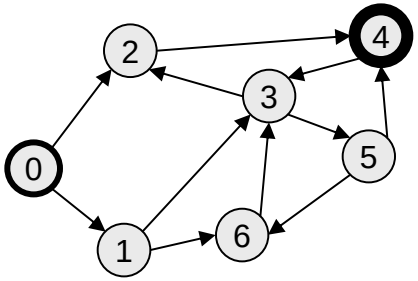



**Procura termina**: a solução corresponde à sequência de nós (estados e operadores) no ramo da árvore de procura que contém o nó correspondente ao estado objectivo

# MÉTODOS DE PROCURA

## Procura em largura (BFS - *Breadth-First Search*)

- Estratégia de controlo
  - Explorar primeiro os nós mais **antigos**




Grafo do
Espaço de Estados

Fronteira de exploração [ ]

[0]

0 [ ]

[**1, 2**]

1 [2]

[2, **3, 6**]

2 [3,6]

[3, 6, **4**]

3 [6,4]

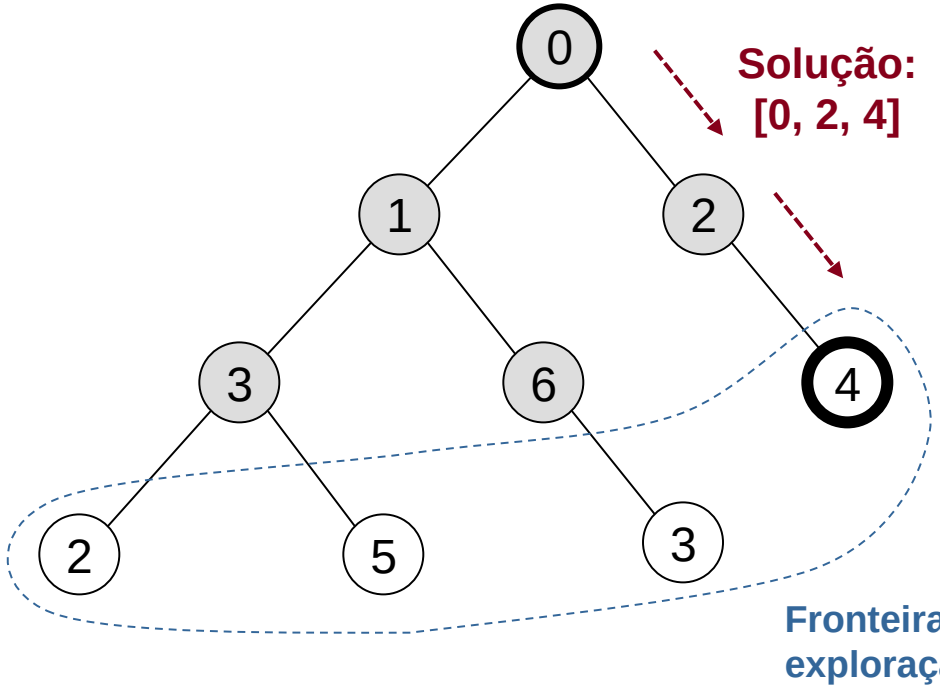
[6,4,**2,5**]

6 [4,2,5]

[4,2,5,**3**]

**Estado objectivo** → **4** [2,5,3]

Árvore de Procura




**Procura termina**: a solução corresponde à sequência de nós (estados e operadores) no ramo da árvore de procura que contém o nó correspondente ao estado objectivo

# PROCURA MELHOR-PRIMEIRO (*BEST-FIRST*)

Método de procura que utiliza uma avaliação de estado baseada no custo para atingir cada estado a partir de um estado inicial

- **Utiliza uma função de custo *f*** para avaliação de cada nó *n* gerado
  - $f(n) \geq 0$
  - $f(n)$ pode ser uma estimativa do custo da solução através do nó *n* (procura informada)
    - **Quanto menor o valor de $f(n)$ mais promissor é o nó *n***
- A fronteira de exploração é ordenada por ordem crescente de $f(n)$

# PROCURA MELHOR-PRIMEIRO

Algoritmo de procura melhor-primeiro (*Best-First Search*)

## Processo de procura

- Criar nó inicial
- Iniciar fronteira com prioridade
- Iniciar explorados
- Enquanto fronteira não vazia
  - Remover primeiro nó da fronteira
  - Se estado do nó é objectivo retornar solução
  - Senão
    - Expandir o nó
    - Por cada nó sucessor
      - Obter estado do nó
      - Se nó ainda não foi explorado ou se é melhor
        - Juntar nó aos explorados
        - Juntar nó à fronteira
- Caso fronteira vazia indicar que não existe solução

# PROCURA MELHOR-PRIMEIRO

## Algoritmo de procura melhor-primeiro (*Best-First Search*)

*Problema*: Definição do problema (estado inicial, operadores, função de teste de objectivo)
*f*(*n*): Função de avaliação de nós

```
1. function procura_melhor_prim(problema, f): Solucao
2.   no ← No(problema.estado_inicial)
3.   fronteira ← FronteiraPrioridade(no, f)
4.   explorados ← {no.estado: no}
5.   while not fronteira.vazia do
6.    no ← fronteira.remover()
7.    if problema.objectivo(no.estado) then
8.      return Solucao(no)
9.    for no_sucessor in expandir(problema, no) do
10.     estado ← no_sucessor.estado
11.     if estado not in explorados or
12.        no_sucessor.custo < explorados[estado].custo then
13.       explorados[estado] ← no_sucessor
14.       fronteira.inserir(no_sucessor)
15.return none
```

# PROCURA MELHOR-PRIMEIRO

Expandir nó (gerar nós sucessores de um nó)

## Processo de expansão de nós

- Iniciar lista de sucessores vazia
- Obter estado do nó
- Para cada operador do problema
  - Gerar estado sucessor aplicando o operador ao estado actual
  - Se nó sucessor existir
    - Calcular custo do nó sucessor (custo até ao nó antecessor + custo da transição para o estado sucessor)
    - Gerar nó sucessor
    - Juntar nó sucessor à lista de nós sucessores
- Retornar lista de nós sucessores

# PROCURA MELHOR-PRIMEIRO

## Expandir nó (gerar nós sucessores de um nó)

```
1.  function expandir(problema, no) : Lista<No>
2.    sucessores ← Lista<No>()
3.    estado ← no.estado
4.    for operador in problema.operadores do
5.       estado_suc ← operador.aplicar(estado)
6.       if estado_suc is not none then
7.          custo ← no.custo + operador.custo(estado, estado_suc)
8.          no_successor ← No(estado_suc, operador, no, custo)
9.          sucessores.juntar(no_successor)
10.   return sucessores
```

# MÉTODOS DE PROCURA INFORMADA

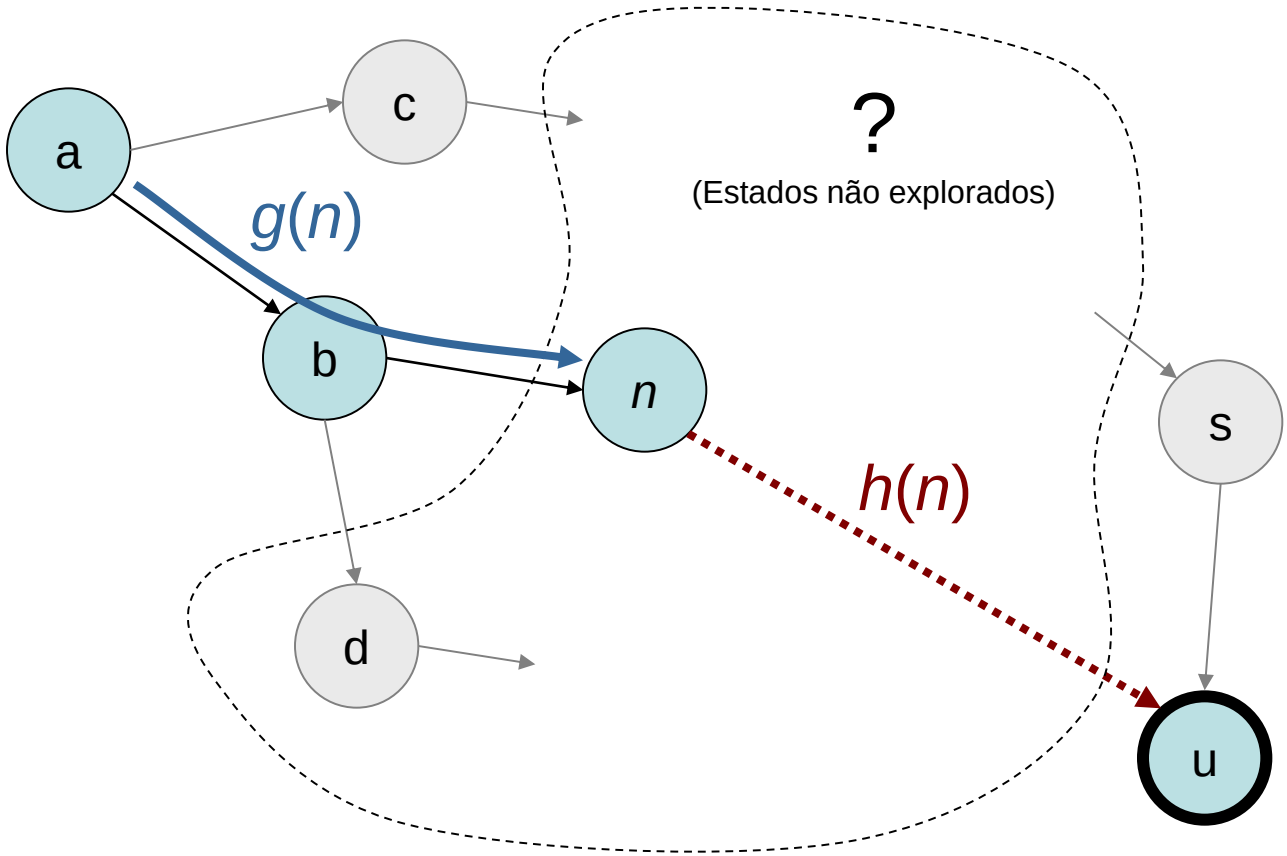
- **Procura informada**
  - Estratégias de exploração do espaço de estados (controlo da procura) **tiram** partido de conhecimento do domínio do problema para **ordenar** a **fronteira de exploração**
  - **Avaliação de estado**
    - Estado é avaliado de acordo com o conhecimento do domínio do problema
    - **Custo de atingir um estado** a partir do estado inicial
      - Custo do percurso explorado
    - **Estimativa de custo para atingir o objectivo** a partir de um estado
      - **Baseado numa *função heurística***

# FUNÇÃO HEURÍSTICA *h*(*n*)

- Representa uma **estimativa do custo de um percurso desde o nó *n* até ao nó objectivo**
- Reflecte conhecimento acerca do domínio do problema
- O seu valor é independente do percurso até *n*
  - Depende apenas de:
    - **Estado** associado a *n*
    - **Objectivo**
- Possível o cálculo de *h*(*n*) de forma eficiente

# MÉTODOS DE PROCURA INFORMADA

***f*(*n*)** baseada em dois tipos de funções de avaliação de custo ***g*(*n*)** e ***h*(*n*)**




***g*(*n*)**: Custo do percurso até ***n***

Custo calculado para a partir do estado inicial atingir o estado ***n***

***h*(*n*)**: Estimativa de custo de ***n*** até ao objectivo

Estimativa de custo (heurística) para a partir do estado ***n*** atingir o estado objectivo ***u*** (sem que o percurso respectivo tenha sido explorado)

# PROCURA MELHOR-PRIMEIRO (*BEST-FIRST*)

## Variantes principais:

- $f(n) = g(n)$
  - **Procura de Custo Uniforme**
    - Não tira partido de conhecimento do domínio do problema
- $f(n) = h(n)$
  - **Procura Sôfrega (*Greedy Search*)**
    - Não tem em conta o custo do percurso explorado
    - Minimização de custo **local**
- $f(n) = g(n) + h(n)$
  - **Procura A*** (heurística admissível)
    - Minimização de custo **global**

# PROCURA SÔFREGA (*GREEDY SEARCH*)

## Função de avaliação de estado $f(s)$:

- Custo estimado a partir de ***s*** até ao objectivo

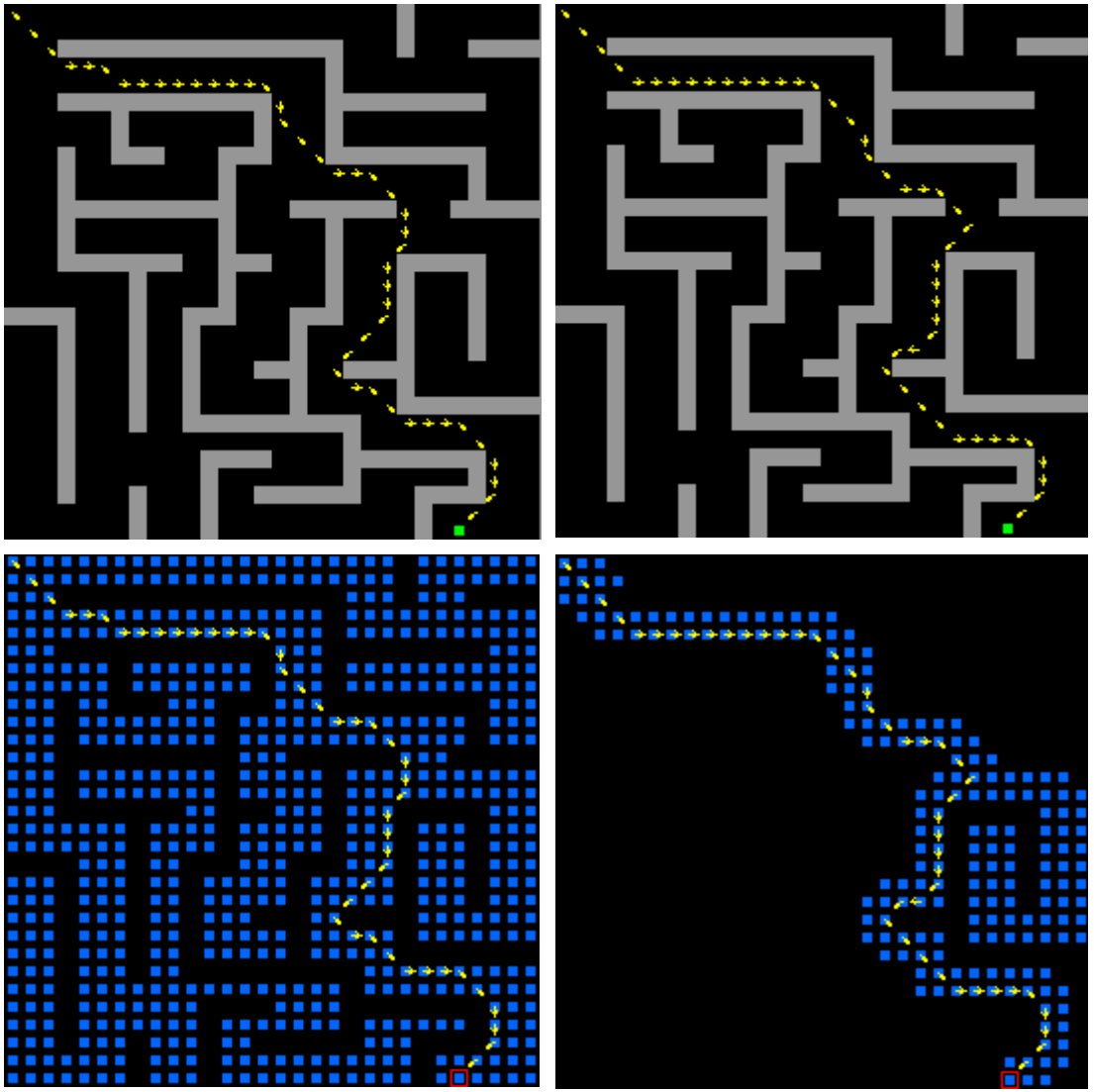
$$f(s) = h(s)$$

## Análise local dos nós expandidos

- Minimização de custo local até ao objectivo
- Pode produzir soluções sub-óptimas

# PROCURA MELHOR-PRIMEIRO (*BEST-FIRST*)

Procura de
custo uniforme
(solução óptima)



Procura sôfrega
(solução sub-óptima)

# PROCURA A*

## Função de avaliação de estado $f(s)$:

- Custo estimado da solução através do estado ***s***

$$f(s) = g(s) + h(s)$$

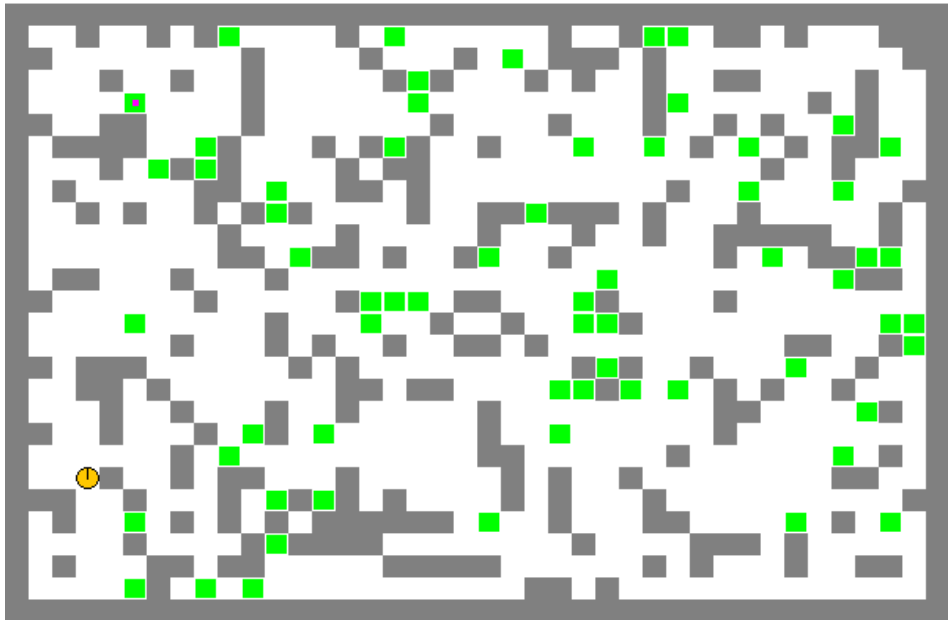
## Análise global dos nós expandidos

- Minimização do custo global até ao objectivo
- **Soluções óptimas**

# PROCURA A*

- **Heurística admissível**
  - $0 \leq h(n) \leq h^*(n)$
  - $h^*(n)$
    - Custo efectivo mínimo do nó *n* até ao objectivo (percurso óptimo)
- Uma **heurística admissível** é ***optimista***
  - A estimativa de custo é sempre inferior ou igual ao custo efectivo mínimo
  - Para um nó objectivo $n_{obj}$
    - $h(n_{obj}) = 0$

# PROCURA A*



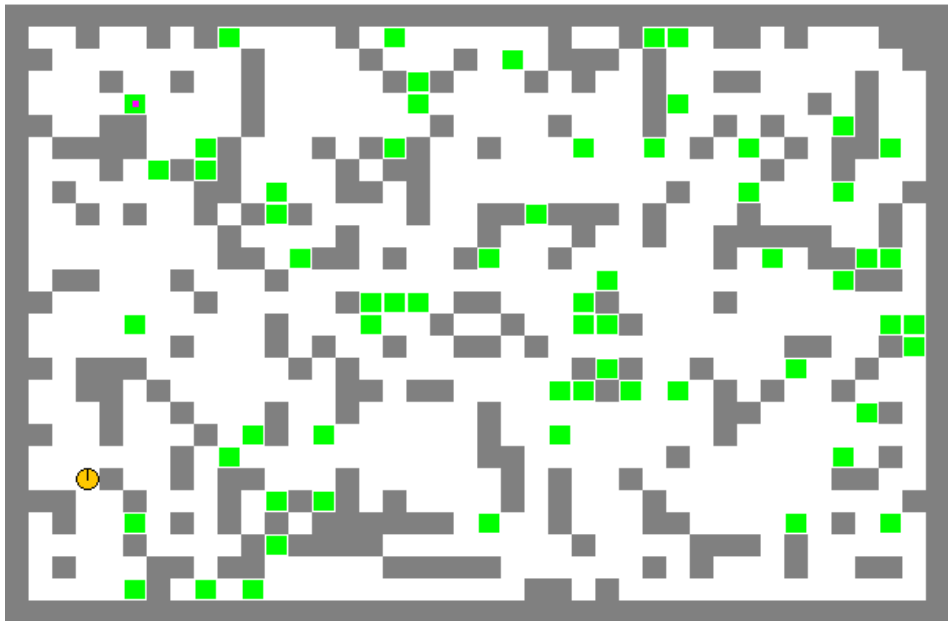
$h_1$ – Distância Euclidiana

$$h_1(n) = \sqrt{(x_n - x_{obj})^2 + (y_n - y_{obj})^2}$$

Admissível?

SIM

# PROCURA A*



$h_2$ – Distância de Manhattan

$$h_2(n) = |x_n - x_{obj}| + |y_n - y_{obj}|$$

## Admissível?

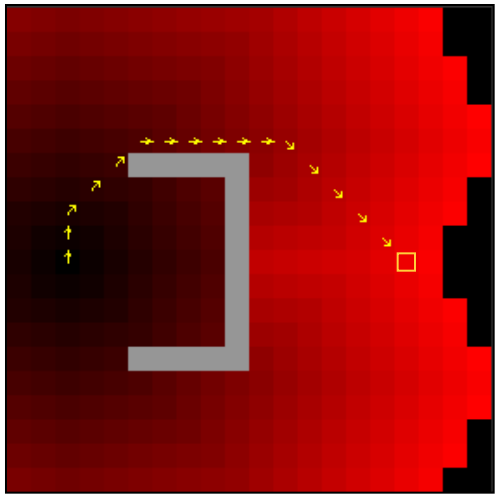
- SIM : Se não forem possíveis movimentos diagonais
- NÃO : Caso contrário

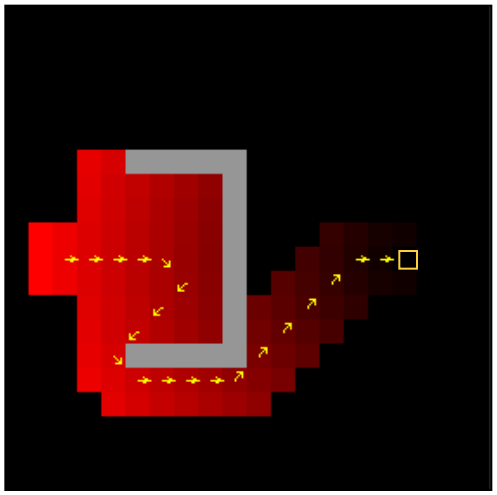
# PROCURA A*

- Método de procura de **eficiência óptima**
  - Nenhum outro algoritmo expandirá menos nós, mantendo as características de ser **completo** e **óptimo**, excepto nas situações de escolha entre nós com $f(n) = C^*$
- **No entanto, não resolve o problema da complexidade combinatória**
  - O número de nós expandidos até ao nó objectivo continua a ser uma **função exponencial** da dimensão do percurso até ao objectivo
  - Função heurística reduz o âmbito de procura
    - Pode não ser suficiente para problemas de grande complexidade

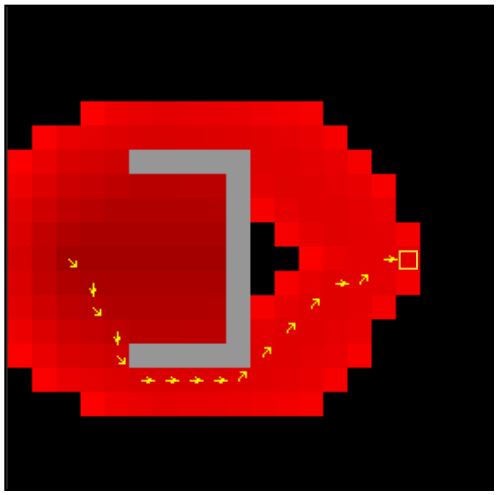
# PROCURA EM ESPAÇOS DE ESTADOS
## COMPARAÇÃO DE MÉTODOS DE PROCURA

Procura de custo uniforme (solução óptima)





Procura sôfrega (solução sub-óptima)



Procura A* (solução óptima)

# PROCURA MELHOR PRIMEIRO - IMPLEMENTAÇÃO

Iniciar mecanismo de procura – criar estruturas de memória de procura:

- Fronteira de exploração
  - Fila com prioridade, para exploração do espaço de estados com base na prioridade associada a cada nó $\boldsymbol{n}$, determinada pela função $\boldsymbol{f(n)}$
- Memória de nós explorados
  - Dicionário indexado por estado, para acesso a nós anteriormente explorados e mantidos em memória para evitar ciclos na procura

```
1. function iniciar()
2.    fronteira = FronteiraPrioridade()
3.    explorados = Dict<Estado:No>
```

# PROCURA MELHOR PRIMEIRO

Resolver problema gerando uma solução se existir

```
1. function resolver(problema): Solucao
2.    limpar_memoria()
3.    no = No(problema.estado_inicial)
4.    memorizar(no)
5.    while not fronteira.vazia:
6.      no = fronteira.remover()
7.      if problema.objectivo(no.estado):
8.         return gerar_solucao(no)
9.      else:
10.        for no_sucessor in expandir(problema, no):
11.           memorizar(no_sucessor)
12.   return none
```

Solucao: List<No>

# PROCURA MELHOR PRIMEIRO

Limpar memória de procura

```
1. function limpar_memoria()
2.    fronteira.limpar()
3.    explorados.clear()
```

# PROCURA MELHOR PRIMEIRO

Memorizar nó, inserindo nó na fronteira de exploração e na memória de nós explorados

```
1. function memorizar(no)
2.    if no.estado not in explorados or
3.       no.custo < explorados[no.estado].custo:
4.    fronteira.inserir(no, f(no))
5.    explorados[no.estado] = no
```

# PROCURA MELHOR PRIMEIRO

Expandir nó gerando os nós sucessores por aplicação dos vários operadores definidos

```
1. function expandir(problema, no): List<No>
2.    sucessores = []
3.    estado = no.estado
4.    for operador in problema.operadores:
5.      estado_suc = operador.aplicar(estado)
6.      if estado_suc:
7.        custo = no.custo +
8.                operador.custo(estado, estado_suc)
9.        no_sucessor = No(estado_suc, operador, no, custo)
10.        sucessores.append(no_sucessor)
11.   return sucessores
```

```
1. interface Operador
2.    aplicar(estado): Estado
3.    custo(estado, estado_suc): float
```

# PROCURA MELHOR PRIMEIRO

Gerar solução a partir do nó final, correspondente a um estado objectivo

- Os nós do ramo da árvore de procura correspondente à solução, que termina com o nó final, são inseridos no início da lista de nós da solução para que a solução fique ordenada por ordem de passos a realizar

```
1. function gerar_solucao(no_final): Solucao
2.    solucao = []
3.    no = no_final
4.    while no:
5.       solucao.insert(0, no)
6.       no = no.antecessor
7.    return solucao
```

Inserir nó no início da lista

```
Solucao: List<No>
```

# NÓ (DA ÁRVORE DE PROCURA)

## Iniciar nó da árvore de procura

```
1. function iniciar(estado,
2.                  operador = none,
3.                  antecessor = none,
4.                  custo = 0)
5.    set estado = estado
6.    set operador = operador
7.    set antecessor = antecessor
8.    set custo = custo
9.    if antecessor:
10.     set profundidade = antecessor.profundidade + 1
11.   else:
12.     set profundidade = 0
```

**set** propriedade = valor (atribuição de valor a propriedade)

# FRONTEIRA COM PRIORIDADE

Mantém os nós abertos por ordem de prioridade

```
1. function iniciar()
2.    nos = PriorityQueue<No>()
```

```
1. function limpar()
2.    nos.clear()
```

```
1. function vazia(): boolean
2.    return nos.empty()
```

```
1. function inserir(no, prioridade)
2.    nos.insert(no, prioridade)
```

```
1. function remover(): No
2.    return nos.remove()
```

# PROCURA DE CUSTO UNIFORME

Função de custo: $f(n) = g(n)$

```
1. function f(no): float
2.    return no.custo
```

# PROCURA SÔFREGA

Função de custo: $f(n) = h(n)$

```
1. function f(no): float
2.    return problema.heuristica(no.estado)
```

# PROCURA A*

Função de custo: $f(n) = g(n) + h(n)$

```
1. function f(no): float
2.    return no.custo + problema.heuristica(no.estado)
```

# BIBLIOGRAFIA


[Russel & Norvig, 2010]

S. Russell, P. Norvig, *Artificial Intelligence: A Modern Approach*, 3rd Edition, Prentice Hall, 2010

[Nilsson, 1998]

N. Nilsson , *Artificial Intelligence: A New Synthesis*, Morgan Kaufmann 1998

[Luger, 2009]

G. Luger , *Artificial Intelligence: Structures and Strategies for Complex Problem Solving* , Addison-Wesley, 2009

[Jaeger & Hamprecht, 2010]

M. Jaeger, F. Hamprecht, *Automatic Process Control for Laser Welding*, Heidelberg Collaboratory for Image Processing (HCI) , 2000

[Pearl, 1984]

J. Pearl, *Heuristics: Intelligent Search Strategies for Computer Problem Solving*, Addison-Wesley, 1984

[Montemerlo, 2008]

M. Montemerlo *et al.*, *Junior: The Stanford Entry in the Urban Challenge*, Stanford Artificial Intelligence Lab, 2008

[Nagy et al., 2018]

D. Nagy, D. Zhao, D. Benjamin, “Nature-Based Hybrid Computational Geometry System for Optimizing Component Structure”, Humanizing Digital Reality, Springer Nature, 2018